

INF-1400: Object-oriented programming

Mandatory assignment 3

Jens Christian Valen Leynse

April 20th, 2017

1 Introduction

The assignment was to create a mayhem clone, which is a game where two spaceships try to take each other down, while surviving all obstacles. The code is written in python 3, and the code includes the requirements of the assignment implemented with classes and methods.

1.1 Requirements

1. The two spaceships shall have four controls: rotating left and right, thrust for propulsion, and fire which should shoot a projectile.
2. Minimum one obstacle in the game world.
3. The ships shall be able to collide with obstacles, the walls and each other.
4. A gravity shall affect the ships.
5. Each ship shall have a score that increases and decreases according to if the player shoots the opponent down, or if the player crashes or gets shot down.
6. Each ship shall have a limited amount of fuel. This fuel decreases so that the player will have to refuel at a landing pad or at a refueling object placed on the screen.

2 Technical Background

The classes are all connected through a has-a relationship to send the information around. This is done through instances where a class is called from within another class. It's done once to connect every class, except for one class which is instantiated several time into a list. Three python libraries are included in the code; pygame, random and time.

3 Design & Implementation

The code is separated into ten classes, the two most prominent being the spaceship class and the game class. An easy way to separate them is to set them into two groups where one interacts with the spaceships, while the other doesn't. They all have something to draw, whether it be a sprite or something else, but the group that interacts affect the player in some way. The size of which every object is drawn is set by a function from the pygame library which rotates and scales the image/sprite.

The first group which interacts, represented by the object file, consists of the classes:

The obstacle class contains an image which is a giant upside-down thumb that moves through the screen in a falling fashion. The position it is falling at is random, but not outside a certain point of the screen to make the refueling stations a safe point from it. When the object is spawned it is added to a list which is iterated through to blit the image at the right coordinate. After a certain time from its addition to the obstacle list, it is removed and re added to the list with a different coordinate through an update method. The timing of which it is respawned, is determined by a delay.

The landing pad class consists of only the image of the landing pad (or the stand which is more concise with the image) and the position it is located at. The image is blitted onto the screen during the game loop.

The fuel sheet class consists of the refueling item that the ship can use to stay afloat. Since the theme of the display in the game is an instrument play-off the refueling item is a note sheet. The position has its own random pattern so that once a note sheet has been collected it will move randomly elsewhere on the screen. As the class before it the image is blitted onto the screen from the game loop.

The second group represented by the classes file, consists of the classes:

Explosion is the class containing the animation for the death sequence. Here ten different images are added to a list which is iterated through one image at the time. There are two Boolean statements connected to the state of the spaceship and explosion. If the spaceship crashes or dies in any way, a trigger is sent to the explosion class and the animation sequence starts. The way this is done is by adding a variable that counts one for each time called from the game loop, a delay is set for the animation to be more fluent. After each count the next image is blitted onto the display. After the first count the second statement of the class which determines if the ship is alive is turned off, and the ship is removed from the screen. After the ten images has been shown, the iterator resets and the ship is respawned again.

The score class's main function is to display a simple text showing the two player scores, one for each corner. The font is collected from a library of fonts in the pygame library. The only information necessary is the amount of points to be displayed and the ship that owns that amount of points.

The health bar class's main function is much like the score class to display information. This information being the life of each ship. The positioning and sizing of the bar is determined by drawing rectangles (lives) along a line by the amount of health the player has. If this amount reaches zero, the player is described as dead by a Boolean statement that sends information to the ship saying it is to be destroyed.

The fuel bar class is the last of the classes that exist almost purely for information to the player. Like the two other classes the positioning and sizing of the object is determined, and the remaining amount is checked. If the amount reaches zero, the thrust of the ship stops. The background of the fuel display is put in place by a black rectangle inside a grey one.

Bullets, which as the name suggests, is the class representing the projectiles shot from the spaceships. This class deviates a bit from the statement that this group of classes doesn't interact with the ships, but it is placed in this group because of every class being in a close relation to the spaceship class. The bullets are displayed as music notes being shot out from the ship. The direction is set as whatever the spaceships vector is at the time of the shot, as well of the positioning. The class is also quite special, in that every method is individual. No method is connected to the object which is self except for the initializer. The only relation in common is the class global bullet list. This list is iterated through to add movement and to be blitted. Each image is differently displayed because of a fill method that colours the image in random colours for each bullet.

The spaceship class which is the biggest contains the information for the player for each event that is to happen and can happen. The class initializer is sent several pieces of information; the model which is displayed, the position, the image, the size and the controls used to steer the spaceship. All the starting values is set and the classes which relates info is instantiated. Here are all the methods:

Draw: Checks the life state of the ship. If the ship dies the image stops being blitted and all the starting values are reset.

Move: Checks the controls for rotation to turn the ships image. A vector is then created to rotate from the ships position so that this vectors endpoint is a position the spaceship can move against. The vector is determined through mathematical functions and if the ships position gets outside of the screen the player explodes and loses points.

Controls: instantiates firstly the move function before checking if the player is pressing the key to thrust. If the key is pressed, the ship is moved towards the vector determined in move and the amount of fuel declines. Should it reach zero, the thrust will stop working. If the key to shoot is pushed, the bullet is continuously instantiated as long as the key is held down and a delay has passed.

Gravity: Simply adds a downwards facing vector to the ships position.

Land: Checks if the ships radius connects with the stand on the screen. If they connect the ships position and speed gets frozen and the fuel refilled.

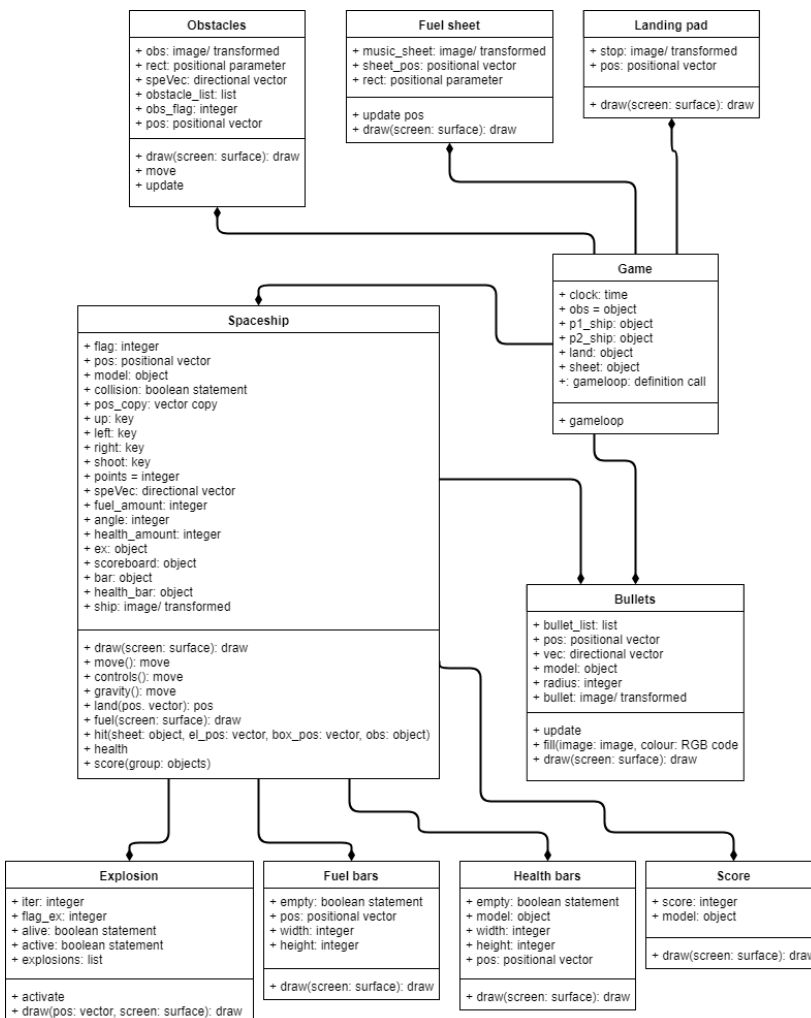
Hit: takes in all the obstacles that interacts with the player and checks if the ship radius connects with any of them. Here most of the information for the health and score points are added.

Fuel: Fills in the fuel display bar with the amount of fuel each player has. It also updates the fuel for changes.

Health: Updates the health bar and runs checks to see if player still alive.

Score: Adds points if the opponent ships health bar reaches empty and makes sure that the points doesn't go below zero while updating the score.

Lastly the game class. This class instantiates the remaining classes, that being the group that interacts. The screen and background are set, and the loop that keeps the game running, with exit options, is placed here. Every object from the file gets drawn and both ships go through an iteration of their methods to activate them all, the bullets are also updated and drawn. Lastly the fps of the game is set, and the display updated.



As shown in the UML diagram, the spaceship class and the game class instantiate the other classes.

There is a configuration file which contains all the constants used in the code that is importing to all the files.

5 Discussion

The way that the classes is instantiated and no-one of the classes inherits any information was not planned, but a way to send the information around was easy this way so the design choice was kept. This however led to some complications regarding the information split between the two players. Most I was able to work around but the fuel bar background got drawn above the fuel amount this way, so a slight deviation in the implementation was made, comparing it to the health and score implementation.

For a long time as well, the rotation of the ship was somehow related to the ships location on the screen. This turned out to be a simple fix because the position of the ship got altered slightly by the placement of an extra position not necessary.

A last-minute alteration of the code was added as well where the size of all the objects is dependent on the size of the screen.

Because of my framework and design choice I ended up not using the pygame sprite class. This I was allowed to do by talking it over with a tutor.

6 Conclusion

All of the requirements for the assignment is met. The spaceships move individually and can lose points by crashing or dying, they gain points by destroying the opponent. The game runs smoothly.

References

The pygame library with its many functions included:

pygame docs. 2018. *pygame dozumentation*. [ONLINE] Available at: <https://pygame.org/docs/>. [Accessed 20 April 2018].

The tool to make an UML-diagram:

draw. 2018. *Mayhem_UML*. [ONLINE] Available at: <https://www.draw.io/>. [Accessed 19 April 2018].

The book of this course:

Philips, D., 2015. *Python 3 Object-oriented Programming*. 2nd ed. Birmingham: Packt Publishing Ltd.